

Resumo do Artigo "Design by Contract"

O artigo de Bertrand Meyer introduz uma das metodologias mais influentes da engenharia de software moderna: o **Design by Contract (DbC)**. A premissa central é que a interação entre diferentes componentes de software deve ser regida por "contratos" formais, semelhantes aos acordos jurídicos entre pessoas ou empresas.

1. O Conceito de Contrato

Para Meyer, um sistema de software não é apenas um conjunto de funções, mas uma rede de fornecedores e clientes. O contrato define as obrigações e os benefícios de cada parte:

- **Obrigações:** O que o cliente deve garantir antes de chamar um serviço.
- **Benefícios:** O que o fornecedor garante entregar se o cliente cumprir sua parte.

2. Os Três Pilares: Pré-condições, Pós-condições e Invariantes

O DbC é implementado através de três tipos de asserções lógicas:

- **Pré-condições (require):** São os requisitos necessários para a execução de um método. Se o cliente chama uma função sem satisfazer a pré-condição, a culpa do erro é do **cliente**.
- **Pós-condições (ensure):** São as garantias que o método oferece ao finalizar. Se a pré-condição foi aceita mas a pós-condição falhou, a culpa é do **fornecedor (desenvolvedor do método)**.
- **Invariantes de Classe:** São condições que devem permanecer verdadeiras durante toda a vida do objeto (exceto durante a execução interna de um método). Elas definem o "estado de saúde" do objeto.

3. Confiabilidade e Robustez

Meyer argumenta que o DbC é superior à "Programação Defensiva" tradicional. Enquanto a programação defensiva incentiva verificações redundantes em todo o código (o que torna o sistema inchado e difícil de manter), o DbC define exatamente quem é responsável por cada verificação. Isso reduz a complexidade e aumenta a **corretude** do sistema.

4. Herança e o Princípio da Substituição

Um ponto crucial do artigo é como os contratos se comportam na herança. Meyer estabelece que uma subclasse pode **enfraquecer a pré-condição** (ser mais tolerante com os inputs) ou **fortalecer a pós-condição** (entregar mais do que o prometido), mas nunca o contrário. Isso garante que o polimorfismo não quebre as expectativas de quem usa a classe base.

Página 2: Aplicação em um Projeto Fictício (Sistema Bancário Digital)

Para ilustrar a aplicação do DbC, utilizaremos um projeto fictício de um **Sistema de Gestão de Contas Bancárias**.

1. Definindo o Contrato da Classe ContaCorrente

No desenvolvimento deste projeto, não usáramos apenas if/else genéricos, mas sim asserções que definem o contrato da classe.

- **Invariante da Classe:**
 - `saldo >= limite_especial_negativo` (O saldo nunca pode ser inferior ao limite permitido).
 - `titular != null` (Toda conta deve ter um dono).

2. Aplicação no Método sacar(valor)

A implementação do contrato para a operação de saque seria:

- **Pré-condições (require):**
 - O valor solicitado deve ser maior que zero (`valor > 0`).
 - O cliente deve ter saldo + limite suficiente para o saque (`valor <= saldo + limite`).
- **Corpo da Operação:**
 - `saldo = saldo - valor`
- **Pós-condições (ensure):**
 - O novo saldo deve ser igual ao `saldo_anterior - valor`.
 - O sistema deve garantir que o saldo final respeite a invariante.

3. Cenário de Falha e Responsabilidade

Imagine que um desenvolvedor da interface (UI) permite que um usuário tente sacar um valor negativo.

- **Sem DbC:** O método sacar poderia processar o valor, resultando em um erro de lógica difícil de rastrear.
- **Com DbC:** A pré-condição valor > 0 falharia imediatamente. Como se trata de uma pré-condição, o log indicaria que o erro está na **chamada** (na UI), e não no motor financeiro. Isso isola o bug instantaneamente.

4. Benefícios Práticos no Projeto

1. **Documentação Automática:** O contrato serve como a melhor documentação técnica para outros desenvolvedores da equipe.
2. **Testes Facilitados:** As pós-condições funcionam como oráculos de teste em tempo de execução.
3. **Manutenibilidade:** Ao herdar de ContaCorrente para criar uma ContaPoupanca, as regras de negócio (invariantes) são herdadas automaticamente, impedindo que novas funcionalidades corrompam regras financeiras essenciais.

Dessa forma, o projeto deixa de ser uma pilha de códigos defensivos e se torna um sistema robusto onde cada componente "sabe" exatamente o que esperar e o que entregar.